

Informe final: Criterios de usabilidad y accesibilidad

Proyecto: Ekko

Asignatura: Usabilidad y Accesibilidad Web

Curso: 2025-2026

Grupo/autores: Carlos Laborda Martinez, Sergio Pernas Gomez, Jonathan Hudrea Colceriu

Fecha: 18/05/2026

1. Portada

Este documento recoge el informe final de la Unidad 3 del proyecto Ekko, centrado en los criterios de usabilidad y accesibilidad aplicados durante el desarrollo de la plataforma. La revisión se ha realizado sobre el repositorio del proyecto, teniendo en cuenta la estructura real de la aplicación: frontend Angular, backend Express, persistencia en MongoDB, almacenamiento de recursos multimedia y despliegue en Vercel.

El informe está redactado para servir como memoria académica y como documento de revisión del producto. Por ese motivo no se limita a enumerar principios teóricos, sino que relaciona cada criterio con decisiones concretas tomadas en la interfaz, en la navegación, en los formularios, en los servicios de datos y en la organización visual de Ekko.

2. Introducción

Ekko es una plataforma web orientada a la búsqueda, visualización, gestión y publicación de contenidos multimedia. La aplicación se centra especialmente en frases, escenas y fragmentos de audio o vídeo relacionados con películas, series, videojuegos y efectos sonoros. El usuario puede descubrir contenidos, filtrarlos por categoría o formato, abrir una vista de detalle, reproducir el fragmento, guardarlo, valorarlo, compartirlo, descargarlo o publicar nuevos elementos si tiene sesión iniciada.

Aunque el proyecto nace dentro de una asignatura universitaria, se ha trabajado con enfoque de producto operativo. Esto significa que no se ha planteado como una maqueta aislada, sino como una aplicación con rutas reales, autenticación, conexión a backend, base de datos MongoDB, subida de archivos, persistencia de preferencias de accesibilidad y una estructura de navegación que cubre distintos escenarios de uso. La aplicación no solo muestra pantallas: permite realizar tareas completas.

El objetivo de este informe es documentar las decisiones de usabilidad y accesibilidad aplicadas en Ekko. Para ello se revisan los principios de interacción, la claridad de los flujos, la adaptación responsive, los mecanismos de feedback y las directrices mínimas de accesibilidad exigidas. También se incluye una evaluación crítica con mejoras futuras, ya que una revisión profesional debe reconocer tanto lo que está resuelto como los puntos que todavía pueden evolucionar.

3. Descripción general de la plataforma

Ekko permite localizar contenidos multimedia mediante búsquedas, filtros y navegación por secciones. La pantalla de inicio presenta contenido destacado y un buscador principal. La sección Discover ofrece filtros por categoría, formato y producción. La sección Library permite consultar los contenidos guardados por el usuario autenticado. La vista Detail concentra la reproducción del fragmento y las acciones asociadas. La página Publish permite publicar nuevos contenidos con metadatos, portada y archivo multimedia. Profile y Settings completan la gestión de cuenta, estadísticas personales y ajustes de accesibilidad. Además, el rol administrador puede acceder a una pantalla específica para revisar publicaciones, añadir o retirar transcripciones y eliminar contenidos.

En la revisión final del 18/05/2026 se reforzaron varios flujos clave. Discover incorpora limpieza de filtros y un selector de producción accesible por teclado; Login y Register asocian estados de error y disponibilidad a sus campos; Publish usa controles de archivo accionables con teclado y mensajes con roles accesibles; Profile mejora el estado de subidas; Settings elimina acciones vacías y declara mejor el despliegue del formulario de cuenta. También se amplió la base semilla y se añadió un script de usuarios demo para preparar la defensa con datos consistentes.

La aplicación contempla tres perfiles principales de usuario. El usuario invitado puede explorar contenidos, buscar, filtrar y consultar la vista de detalle. El usuario autenticado puede, además, guardar publicaciones, valorar fragmentos, descargar medios, subir nuevos contenidos, editar su perfil y configurar ajustes visuales. El usuario administrador añade tareas de mantenimiento sobre todas las publicaciones, especialmente la gestión de transcripciones accesibles y la eliminación de contenidos. Esta separación se refleja en la lógica de navegación: páginas como Library y Publish redirigen al login si no existe token de sesión, mientras que Admin exige un rol específico.

Las funcionalidades principales implementadas son las siguientes:

- búsqueda de contenidos desde Home y filtrado avanzado desde Discover;
- visualización de resultados mediante tarjetas con título, frase, formato, valoración y visitas;
- gestión de usuarios con registro, inicio de sesión, perfil, avatar, estadísticas y cambio de contraseña;
- subida y gestión de contenidos desde Publish, con validación de campos y límites de tamaño;
- gestión administrativa de publicaciones, transcripciones y borrado de contenidos;
- conexión con backend Express mediante servicios Angular centralizados;
- persistencia de usuarios, publicaciones, guardados y valoraciones en MongoDB;
- subida de archivos y portadas a Cloudinary desde el backend;
- diseño responsive con barra inferior en móvil y navegación lateral en escritorio;
- ajustes de accesibilidad persistentes, como alto contraste, tamaño de texto, reducción de movimiento, controles grandes, subrayado de enlaces y tipografía legible.

Desde el punto de vista técnico, el frontend se organiza en componentes y páginas Angular dentro de `frontend/src/app`. Las llamadas a la API se centralizan en servicios como `quotes.services.ts`, `auth.service.ts` y `user.service.ts`, que dependen de una base común definida en `api-url.ts`. El backend Express expone rutas bajo `/api/auth` y `/api/quotes`, con controladores separados para autenticación, usuarios y publicaciones. En producción, el frontend usa el prefijo `/_/backend/api`, coherente con la configuración del servicio backend en Vercel.

4. Criterios de usabilidad aplicados

4.1 Facilidad de aprendizaje

La facilidad de aprendizaje consiste en que una persona pueda entender rápidamente qué ofrece la interfaz y cómo empezar a utilizarla sin necesitar instrucciones externas. En Ekko este principio se trabaja mediante una estructura de secciones reconocible: Inicio, Descubrir, Librería, Publicar y Perfil. La navegación principal utiliza iconos acompañados de texto y mantiene siempre el mismo conjunto de destinos.

Una acción concreta implementada es el buscador principal en Home, situado en una zona visible de la pantalla y acompañado de un placeholder descriptivo: "Busca frases icónicas...". El usuario no necesita aprender un lenguaje especial de consulta; puede escribir parte de una frase, una obra, un personaje o un elemento relacionado, y la aplicación recalcula los resultados.

El beneficio para el usuario es que la primera interacción se entiende de forma inmediata. La aplicación comunica que su tarea principal es localizar fragmentos y permite empezar por una búsqueda sencilla antes de entrar en filtros más avanzados.

4.2 Flexibilidad

La flexibilidad se refiere a la posibilidad de alcanzar un objetivo por diferentes caminos y adaptarse a distintos tipos de usuario. Ekko permite explorar desde Home, refinar desde Discover, consultar guardados desde Library o entrar directamente en una publicación mediante su URL de detalle. Además, la aplicación distingue entre usuarios invitados y autenticados sin bloquear toda la experiencia: se puede explorar sin cuenta, pero las acciones personales requieren autenticación.

Una acción concreta implementada es la combinación de búsqueda textual, filtros por categoría, filtros por formato y selección de producción en Discover. También existe ordenación y filtrado en Library, donde el usuario puede limitar sus guardados por formato, categoría o título.

El beneficio es que la aplicación sirve tanto para una exploración rápida como para una búsqueda concreta. Un usuario que solo recuerda una frase puede usar el buscador; otro que quiera encontrar vídeos de una película concreta puede usar los filtros.

4.3 Consistencia

La consistencia permite que la interfaz mantenga reglas visuales y de interacción estables. En Ekko se repiten patrones como cabecera con marca, tarjetas de contenido, botones redondeados, acentos dorados, estados activos mediante `aria-pressed` y navegación principal persistente. Las páginas principales comparten estructura: un `main` con `id="main-content"`, un encabezado o bloque de contexto, una zona de herramientas y una zona de resultados o contenido.

Una acción concreta implementada es el componente `navbar`, reutilizado en Home, Discover, Detail, Library y Publish. Este componente marca la sección activa con `aria-current` y conserva el mismo orden de navegación en móvil y escritorio.

El beneficio para el usuario es la previsibilidad. Una vez aprende dónde está el acceso a Discover, Publish o Perfil, puede trasladar ese aprendizaje al resto de pantallas. Esto reduce errores y hace que la aplicación parezca un producto coherente.

4.4 Robustez

La robustez implica que el sistema responda de forma estable ante datos incompletos, errores de red o estados no ideales. En Ekko se observa tanto en frontend como en backend. Los servicios normalizan datos antes de mostrarlos, el backend valida campos obligatorios y las páginas contemplan estados de carga, error o ausencia de resultados.

Una acción concreta implementada está en `quotes.services.ts`, donde las publicaciones recibidas se normalizan para asegurar valores por defecto en campos como texto, título, año, puntuación, visitas, portada, duración o tipo de medio. En backend, `quote.controller.js` valida que una publicación tenga los campos requeridos antes de guardarla y limita la valoración a números enteros entre 1 y 5.

El beneficio es que la interfaz evita fallos visibles cuando una respuesta llega incompleta o cuando el usuario intenta enviar datos inválidos. La robustez mejora la confianza en la aplicación porque los errores no rompen la navegación.

4.5 Recuperabilidad

La recuperabilidad consiste en facilitar que el usuario corrija una acción o vuelva a un estado válido. En Ekko se aplica mediante botones de vuelta, limpieza de filtros, mensajes de error y redirecciones controladas. Cuando un usuario intenta acceder a Library o Publish sin sesión, la aplicación no muestra una pantalla rota: lo redirige al login.

Una acción concreta implementada es el botón "Limpiar filtros" en Discover y Library. En Discover restaura búsqueda, categoría, formato y producción, y en Library devuelve búsqueda, formato, categoría y ordenación al estado inicial.

En Publish, si el archivo supera el tamaño permitido, se limpia la selección y se muestra un mensaje indicando el límite correspondiente.

El beneficio es que el usuario puede recuperarse sin abandonar la aplicación. Si se equivoca filtrando, vuelve a empezar con un clic; si sube un archivo demasiado grande, entiende por qué no puede continuar.

4.6 Tiempo de respuesta

El tiempo de respuesta está relacionado con la rapidez percibida y con la capacidad de la interfaz para reaccionar a las acciones del usuario. En Ekko, las búsquedas y filtros principales se aplican en cliente sobre la colección ya cargada, lo que evita llamadas repetidas al backend en cada pulsación. Además, `QuoteService` cachea `getQuotes()` con `shareReplay`, reutilizando la misma respuesta para varias páginas.

Una acción concreta implementada es la actualización inmediata de resultados en Home mediante `onSearchChange()`, que recalcula coincidencias localmente y ordena por relevancia, valoración y visitas. También se actualiza localmente la valoración y el contador de visitas en Detail tras recibir confirmación del backend.

El beneficio es una experiencia más fluida. El usuario percibe que el sistema responde al momento, especialmente al buscar y filtrar, que son tareas de uso frecuente.

4.7 Adecuación de las tareas

La adecuación de las tareas significa que cada pantalla debe estar diseñada alrededor de lo que el usuario quiere hacer en ese momento. En Ekko, cada ruta tiene una finalidad clara. Home sirve para descubrir y buscar; Discover para filtrar; Detail para reproducir y actuar sobre un fragmento; Library para gestionar guardados; Publish para crear contenidos; Settings para ajustar accesibilidad y cuenta.

Una acción concreta implementada es la separación visual en Publish entre la columna multimedia y la columna de metadatos en escritorio. El usuario primero prepara el archivo, portada, duración y tipo, y después completa frase, categoría, obra, año, actor, personaje, sinopsis y etiquetas.

El beneficio es que el flujo coincide con la tarea real de publicar un fragmento. La interfaz no mezcla acciones sin relación y permite revisar el contenido antes de enviarlo.

4.8 Disminución de la carga cognitiva

Reducir la carga cognitiva implica no obligar al usuario a recordar información ni interpretar demasiados elementos al mismo tiempo. Ekko usa etiquetas, agrupaciones visuales, estados activos y mensajes contextuales para que cada decisión sea reconocible.

Una acción concreta implementada es el uso de chips y controles segmentados para categorías, formatos, filtros de color y tamaños de texto. Estos controles muestran opciones visibles en lugar de exigir que el usuario escriba valores o recuerde códigos internos como `movie`, `series`, `game` o `sfx`.

El beneficio es que las decisiones se toman por reconocimiento, no por memoria. Esto mejora la accesibilidad cognitiva y reduce errores en procesos como buscar, filtrar o configurar la interfaz.

5. Acciones específicas de usabilidad web

Ekko aplica varias acciones concretas de usabilidad web que se pueden comprobar en sus páginas principales.

La titulación clara de páginas y secciones aparece en encabezados como "Encuentra el fragmento exacto", "Accesibilidad y cuenta", "Prepara el fragmento antes de publicarlo", "Resultados" o "Sinopsis". Estos títulos no son decorativos: ayudan a entender el propósito de cada bloque.

La identificación visual de zonas principales se consigue mediante secciones diferenciadas. Discover separa panel de filtros y resultados. Library separa resumen, herramientas y resultados guardados. Detail distingue reproducción, valoración, acciones, créditos y sinopsis. Publish divide multimedia y metadatos. Esta separación permite escanear la interfaz sin leer todo el contenido.

Los enlaces y botones son reconocibles por su forma, contraste, iconografía y estados de interacción. La navegación principal usa iconos de inicio, descubrir, biblioteca, publicar y perfil. Las acciones de Detail incluyen botones explícitos para descargar, compartir y guardar. Los botones activos de filtros cambian visualmente y exponen estado mediante `aria-pressed`.

La navegación es clara porque se mantiene una barra común. En móvil se sitúa en la parte inferior, cerca del pulgar, y en escritorio se transforma en sidebar lateral. Además, hay botones de vuelta en pantallas como Detail, Discover, Publish, Library, Login y Register.

La jerarquía visual se basa en tamaño, peso y posición. Los títulos de página se muestran como elementos dominantes, los filtros aparecen antes de los resultados y las tarjetas presentan primero la frase, después la obra y finalmente metadatos como formato, visitas o puntuación.

El feedback visual se utiliza en varias acciones: mensajes de error en login y registro, aviso de "Guardando ajustes..." en Settings, estados "Entrando...", "Creando cuenta..." o "Publicando..." en formularios, mensajes de guardado o valoración en Detail y avisos de tamaño de archivo en Publish.

El diseño responsive está incorporado en plantillas y estilos. Se usan `grid`, `flex`, `clamp`, variables CSS de espaciado y media queries. En móvil predominan columnas simples; en tablet se limita el ancho; en escritorio las páginas adoptan layouts de varias columnas.

Los formularios son comprensibles porque agrupan campos relacionados y usan `label`, `autocomplete`, placeholders y validaciones. Login y Register incluyen labels ocultos para lectores de pantalla; Settings usa labels visibles para contraseña y toggles; Publish agrupa campos en bloques de formato, portada, categoría, metadatos y publicación.

6. Criterios de accesibilidad aplicados

6.1 Uso de texto alternativo en elementos gráficos

El texto alternativo permite que una imagen transmita su significado a personas que usan lector de pantalla o que no pueden cargar el recurso visual. En Ekko se aplica en imágenes relevantes como el logotipo (`alt="Ekko"`), portadas de publicaciones (`[alt]="quote.workTitle"`), avatar de perfil (`[alt]="profile.username"`) y vista previa de portada en Publish (`alt="Vista previa de la portada"`).

Esta directriz beneficia especialmente a las tarjetas de Home, Discover y Library, la vista Detail y el perfil de usuario. En términos WCAG 2.1, se relaciona con el criterio 1.1.1 Contenido no textual.

Como mejora recomendada, algunas portadas que se aplican como `background-image` en tarjetas podrían documentarse mejor si en el futuro se convierten en imágenes reales o si se incorpora una descripción textual más específica. Actualmente la tarjeta completa tiene `aria-label` con obra y cita, lo que compensa parte de esa necesidad.

6.2 Identificación del idioma del documento

Indicar el idioma permite que lectores de pantalla, navegadores y herramientas de traducción interpreten correctamente la pronunciación y reglas del contenido. Ekko lo aplica en `frontend/src/index.html` mediante `<html lang="es">`.

Esta decisión beneficia a toda la aplicación, porque Angular renderiza las rutas dentro de ese documento base. Se relaciona con WCAG 2.1, criterio 3.1.1 Idioma de la página.

Como mejora futura, si la aplicación incorporase contenidos en varios idiomas, sería recomendable marcar fragmentos concretos con `lang` cuando el texto de una cita o título estuviera en un idioma distinto al español.

6.3 Validación de la sintaxis de los archivos

La validación sintáctica busca que HTML, CSS y JavaScript/TypeScript estén bien formados para favorecer la compatibilidad con navegadores y tecnologías de apoyo. En Ekko la estructura Angular está organizada en componentes y plantillas. El proyecto compila con `npm run build`, lo que valida TypeScript, plantillas y configuración del frontend. En backend se pueden revisar archivos JavaScript con `node --check`.

Esta directriz beneficia a toda la interfaz, porque una plantilla mal formada puede afectar a la navegación por teclado, a los roles accesibles o a la lectura por tecnologías de apoyo. Se relaciona con el principio de robustez de WCAG.

Como mejora recomendada, sería conveniente añadir al flujo de verificación herramientas automáticas como Lighthouse, axe DevTools o validadores HTML para revisar atributos, contraste y nombres accesibles.

6.4 Alto contraste en textos

El contraste suficiente permite leer el contenido en distintas condiciones visuales. Ekko utiliza una interfaz oscura con textos claros y acentos dorados. Además, incluye un ajuste explícito de "Alto contraste" en Settings. Al activarlo, el servicio `accessibility.service.ts` aplica `data-high-contrast="true"` al elemento `html`, y `styles.css` refuerza fondo, bordes, foco y placeholders.

Esta directriz beneficia a toda la interfaz, especialmente formularios, botones, tarjetas, filtros y navegación. Se relaciona con WCAG 2.1, criterios 1.4.3 Contraste mínimo y 1.4.11 Contraste no textual.

Como mejora futura, conviene realizar una medición formal de contraste con herramientas específicas, porque algunos textos secundarios en gris o azul sobre fondo oscuro pueden necesitar ajustes según el tamaño exacto y el contexto.

6.5 No codificar información exclusivamente mediante color

La información no debe depender solo del color, ya que algunos usuarios no distinguen ciertos tonos. Ekko combina color con texto, iconos, etiquetas y estados. Por ejemplo, los filtros activos usan cambio de color, pero también conservan texto visible y `aria-pressed`. En Register, la disponibilidad de nombre o correo se comunica con mensajes como "Nombre disponible" o "Nombre no disponible", no solo con verde o rojo.

Esta directriz beneficia a filtros, formularios, estrellas de valoración, estados activos y mensajes. Se relaciona con WCAG 2.1, criterio 1.4.1 Uso del color.

Como mejora recomendada, se podría revisar la valoración por estrellas para añadir texto visible más descriptivo cuando sea necesario, aunque ya existe `aria-label` en los bloques de puntuación.

6.6 Uso completo de la interfaz con teclado

Una interfaz accesible debe poder utilizarse sin ratón. Ekko incorpora navegación por teclado en tarjetas interactivas mediante `tabindex="0"` y gestión de `keydown.enter` y `keydown.space` en Home, Discover y Library. En Discover, el selector de producción se puede abrir desde teclado, expone `aria-expanded`, `aria-controls`, `role="listbox"` y `role="option"`, permite moverse con flechas, Home, End y Esc, y devuelve el foco al disparador al cerrarse. También existen estilos globales de `:focus-visible`, skip-link al contenido principal y controles nativos para inputs, selects, botones, audio y vídeo.

Esta directriz beneficia a usuarios de teclado, personas con movilidad reducida y usuarios de tecnologías de apoyo. Se relaciona con WCAG 2.1, criterios 2.1.1 Teclado, 2.4.1 Evitar bloques y 2.4.7 Foco visible.

Como mejora recomendada, se debería revisar con pruebas manuales completas el orden de tabulación en tarjetas que contienen acciones internas, especialmente en Home, donde la tarjeta completa navega al detalle y dentro existe un botón visual de reproducción. La solución más robusta sería evitar controles anidados o definir con claridad qué elemento recibe el foco.

6.7 No uso de tablas para maquetar

Las tablas deben reservarse para datos tabulares, no para construir layouts visuales. Ekko no usa tablas de maquetación en sus plantillas principales. La estructura visual se construye con CSS Grid, Flexbox, componentes, secciones y artículos.

Esta directriz beneficia a toda la aplicación, porque la lectura semántica es más limpia y los cambios responsive son más naturales. Se relaciona con el principio de robustez y con la correcta interpretación de la estructura por tecnologías de apoyo.

Como mejora futura, si se incorporase una sección administrativa con datos en formato tabla, debería usarse una tabla HTML real solo cuando el contenido fuera tabular, con encabezados y asociaciones correctas.

7. Relación con WCAG 2.1

El diseño de Ekko se alinea con los cuatro principios generales de WCAG 2.1: perceptible, operable, comprensible y robusto.

El principio perceptible se trabaja mediante textos legibles, contraste elevado, alternativas textuales en imágenes importantes, etiquetas en controles y una jerarquía visual clara. Las portadas tienen `alt` en Discover, Detail, Library y Profile. Los controles de puntuación y reproducción usan `aria-label`, y la sección Settings permite ajustar tamaño de texto, contraste, filtros de color y tipografía.

El principio operable se refleja en el uso de elementos nativos como botones, enlaces, inputs, selects, audio y vídeo con controles. La aplicación incluye skip-link al contenido principal, estilos de foco visibles y tarjetas accesibles por teclado. La navegación principal tiene `aria-label="Navegacion principal"` y marca el destino activo con `aria-current`.

El principio comprensible se observa en la organización de tareas y en el feedback. Login y Register muestran errores inline asociados a los campos mediante `aria-invalid` y `aria-describedby`; Register comunica disponibilidad de usuario y correo con `role="status"` o `role="alert"`. Publish informa de campos incompletos o archivos demasiado grandes, Settings comunica el guardado de ajustes y Detail muestra mensajes tras valorar, guardar o copiar enlaces. Los formularios usan campos agrupados y textos de ayuda cuando la tarea lo necesita.

El principio robusto se apoya en la estructura Angular, la centralización de servicios HTTP, la normalización de datos, la validación en backend y el uso de roles/atributos ARIA cuando complementan a la semántica. La aplicación identifica el idioma del documento y evita maquetar con tablas, lo que favorece la interpretación por navegadores y tecnologías de asistencia.

8. Responsive design

Ekko está diseñada para funcionar en móvil, tablet y escritorio. En móvil, las pantallas priorizan una columna principal, tarjetas apiladas y barra de navegación inferior. Esta decisión facilita el acceso con el pulgar a las secciones principales y reduce desplazamientos laterales.

En tablet, el ancho se limita mediante variables como `--app-tablet-width`, manteniendo una lectura cómoda. Las tarjetas y formularios conservan separación suficiente, y los controles siguen teniendo tamaños táctiles razonables.

En escritorio, la navegación se transforma en sidebar lateral. Las páginas aprovechan mejor el espacio horizontal: Discover separa filtros y resultados, Library puede mostrar herramientas y resultados en columnas, Detail organiza multimedia, acciones, créditos y sinopsis con grid, y Publish divide el trabajo entre columna multimedia y columna de metadatos.

Entre las decisiones responsive más relevantes destacan:

- barra inferior en móvil y barra lateral en escritorio;
- tarjetas adaptables con `grid` y `flex`;
- espaciados mediante variables CSS y `clamp`;
- tamaños mínimos en controles interactivos;
- formularios de una columna en móvil y varias columnas en escritorio;
- separación visual entre zonas para evitar saturación;
- fuentes base ajustadas y prevención de zoom accidental en inputs móviles con `font-size: 16px`.

Esta adaptación mejora la usabilidad porque no obliga al usuario a interactuar con una versión de escritorio reducida. Cada tamaño de pantalla recibe una distribución adecuada a sus limitaciones.

9. Evaluación crítica y mejoras futuras

Ekko está bien resuelta en varios aspectos. La navegación principal es estable, las rutas cubren tareas reales, la interfaz diferencia secciones, existen controles de accesibilidad persistentes y la aplicación no depende únicamente de datos simulados: conecta con backend, base de datos y almacenamiento externo. También hay un esfuerzo claro en feedback visual y textual, especialmente en formularios, ajustes, publicación y detalle.

La búsqueda y los filtros están planteados de forma útil. Home permite una exploración rápida, Discover ofrece refinamiento y Library resuelve una necesidad distinta: gestionar elementos ya guardados. Esta separación evita mezclar demasiadas tareas en una sola pantalla.

También destaca la configuración de accesibilidad desde Settings. No se limita a un único modo de contraste, sino que incorpora reducción de movimiento, tamaño de texto, controles grandes, subrayado de enlaces y tipografía legible. Estos ajustes se aplican mediante atributos `data-*` en el documento, lo que permite modificar estilos globales de forma consistente.

Una mejora importante sería realizar pruebas reales con usuarios. La aplicación tiene varios flujos relevantes: buscar un fragmento, guardar una publicación, valorar contenido, cambiar ajustes de accesibilidad y publicar un nuevo medio. Observar a usuarios reales permitiría detectar pasos confusos o textos que podrían simplificarse.

En formularios, ya se ha avanzado hacia una validación más descriptiva por campo en Login y Register mediante `aria-invalid`, `aria-describedby` y mensajes con rol accesible. Como mejora futura, convendría extender ese mismo nivel de detalle a otros formularios complejos, especialmente Publish y Settings, indicando de forma específica qué campo falta o qué regla no se cumple.

Para contenido multimedia, se ha añadido una gestión administrativa de transcripciones accesibles. El usuario que publica no introduce transcripción; esa tarea queda reservada al administrador, que puede añadir marcas de tiempo desde la pantalla Admin mientras revisa el audio o vídeo. Cuando existen marcas de tiempo, la vista Detail puede mostrarlas como subtítulos sincronizados sobre el reproductor y también mantiene un bloque de transcripción accesible con las líneas temporales debajo del contenido.

Por último, se recomienda revisar algunos textos internos y mensajes del backend para asegurar codificación correcta de caracteres en español. La interfaz principal utiliza entidades HTML o texto correcto en Angular, pero en algunos

mensajes del backend se aprecian caracteres mal codificados. No afecta al flujo principal si no se muestran directamente, pero conviene corregirlo antes de una entrega final pública.

10. Conclusión

Ekko se ha desarrollado con un enfoque de producto y no como una simple práctica aislada. La aplicación integra frontend, backend, base de datos, autenticación, gestión de contenidos, reproducción multimedia, publicación de archivos y ajustes de accesibilidad. Esta combinación permite evaluar la usabilidad en flujos completos, no solo en pantallas estáticas.

Desde el punto de vista de usabilidad, el proyecto presenta una navegación clara, secciones diferenciadas, búsqueda y filtros adecuados, feedback en acciones del usuario, formularios comprensibles y adaptación responsive. Desde el punto de vista de accesibilidad, incorpora idioma del documento, skip-link, foco visible, alternativas textuales, labels, controles por teclado, alto contraste configurable y ajustes visuales persistentes.

La evaluación también muestra margen de mejora, especialmente en auditorías automáticas, pruebas con lectores de pantalla, validación avanzada de formularios y accesibilidad específica del contenido multimedia. Estas mejoras no invalidan el trabajo realizado; al contrario, muestran que el proyecto tiene una base suficientemente sólida para seguir evolucionando.

En conjunto, Ekko demuestra una aplicación práctica de criterios de usabilidad y accesibilidad web: cuida la claridad visual, reduce la carga cognitiva, facilita la navegación, responde a distintos dispositivos y contempla necesidades de usuarios con diferentes preferencias o limitaciones.

Checklist de verificación

Principio cumplido	Acción aplicada	Evidencia en la interfaz	Mejora pendiente
Facilidad de aprendizaje	Navegación principal estable y buscador visible	Home, Navbar, Discover	Revisar textos con usuarios reales para confirmar comprensión
Flexibilidad	Búsqueda, filtros, biblioteca y acceso directo a detalle	Home, Discover, Library, ruta <code>/quote/:id</code>	Añadir filtros server-side si la colección crece mucho
Consistencia	Reutilización de patrones visuales y navegación	<code>navbar.component</code> , tarjetas, botones, chips	Crear una guía visual formal del sistema
Robustez	Normalización de datos y validaciones backend/frontend	<code>quotes.services.ts</code> , <code>quote.controller.js</code> , Publish	Añadir tests e2e de flujos críticos
Recuperabilidad	Botones de vuelta, limpieza de filtros y mensajes de error	Discover, Library, Publish, Login, Detail	Extender mensajes por campo al resto de formularios complejos
Tiempo de respuesta	Cache con <code>shareReplay</code> y filtrado local	<code>QuoteService.getQuotes()</code> , Home, Discover, Library	Medir rendimiento con colecciones grandes

Adecuación de tareas	Pantallas separadas por objetivo	Home, Discover, Detail, Library, Publish, Settings	Validar flujo de publicación con usuarios no técnicos
Disminución de carga cognitiva	Chips, estados activos, agrupaciones y textos de ayuda	Discover, Library, Settings, Publish	Simplificar textos secundarios donde haya saturación visual
Texto alternativo	alt en logo, portadas, avatar y previews	Home, Discover, Detail, Library, Profile, Publish	Revisar imágenes usadas como fondo en tarjetas
Idioma del documento	lang="es" en HTML base	frontend/src/index.html	Añadir lang por fragmento si hay contenido multilingüe
Sintaxis válida	Compilación Angular y revisión sintáctica posible en backend	npm run build, node --check	Incorporar validadores HTML/a11y en CI
Alto contraste	Ajuste de alto contraste persistente	Settings, accessibility.service.ts, styles.css	Medición formal de contraste WCAG
No depender solo del color	Texto, iconos, aria-pressed y mensajes explícitos	Filtros, Register, Settings, rating	Añadir más descripciones visibles en algunos indicadores
Uso con teclado	Skip-link, foco visible, tarjetas con Enter/Espacio y selector accesible en Discover	app.component.html, styles.css, Home, Discover, Library	Revisar controles anidados dentro de tarjetas
No usar tablas para maquetar	Layout con Grid, Flexbox, secciones y artículos	Plantillas Angular y CSS	Usar tablas solo si aparece contenido realmente tabular
Responsive design	Barra inferior/lateral, grids adaptables y variables CSS	Navbar, Discover, Library, Detail, Publish	Pruebas en dispositivos reales y Lighthouse móvil

Cambios de accesibilidad realizados durante esta revisión

Durante la revisión final sí se incorporaron cambios de código y documentación. Los más relevantes para este informe son:

- **Discover** : selector de producción con soporte de teclado, roles ARIA, cierre con `Esc` , devolución de foco y acción `Limpiar filtros` .
- **Login** y **Register** : campos con `aria-invalid` , mensajes asociados mediante `aria-describedby` y estados anunciados para errores o disponibilidad.

- `Publish` : selección de archivo multimedia y portada mediante botones enfocables, mensajes con `role="alert"` o `role="status"` y copy más claro.
- `Settings` : formulario de cuenta asociado con `aria-controls` y eliminación de una acción de privacidad sin flujo real en la entrega.
- `Profile` : texto contextual y estado vacío más explícito en el bloque de subidas.
- `Detail` : subtítulos sincronizados y bloque de transcripción accesible cuando existen marcas de tiempo.
- `Backend` y `demo` : ampliación de la colección semilla y script `npm run seed:demo-users` para preparar usuarios de prueba.